

Role 3: Database Architecture

Domain: Relational Schema Modeling, Dual-Persistence Engine, SQL Aggregations & Migrations

Core Technology: PostgreSQL • Drizzle ORM • Node-Postgres (pg.Pool) • Drizzle Zod • JSON Document Storage

1. Executive Summary & Scope

The **Database Architect** is responsible for data persistence, schema integrity, and storage resiliency in the SatyaCheck verification platform. The role designs relational models capable of storing complex forensic news analyses, multi-turn conversational transcripts, and statistical aggregate metrics, while providing a zero-downtime dual-persistence fallback strategy.

Core Engineering Objectives:

- Design normalized, high-performance relational schemas using **Drizzle ORM** in pure JavaScript (ES Modules).
- Implement hybrid relational-document storage utilizing PostgreSQL JSONB and native array types for nested forensic data.
- Build a **Dual-Persistence Engine**: Primary storage routes to PostgreSQL; if the database is unavailable, writes failover seamlessly to an atomic local JSON document store (analyses-storage.json) without throwing 500 errors.
- Formulate optimized SQL aggregation queries (unnest, group-by, windowing) for dashboard analytics and trending topics.
- Manage automated schema migrations and validation boundaries with **Drizzle Kit** and **Drizzle Zod**.

2. Complete Technology Stack

Technology	Version	Role & Architectural Rationale
PostgreSQL	v15 / v16	Enterprise ACID-compliant relational engine with advanced JSONB indexing, array operators, and connection pooling.
Drizzle ORM	0.38.4	Zero-overhead, lightweight SQL query builder with native ES Module support and zero binary engine dependencies.
pg (node-postgres)	8.13.1	Battle-tested PostgreSQL client for Node.js featuring robust connection pooling via Pool.
Drizzle Kit	0.30.2	CLI migration runner for schema diffing, automated SQL generation, and instant push synchronization.
Drizzle Zod	0.7.0	Auto-generates runtime request validation schemas directly from Drizzle table definitions.
Local Document Store	Atomic JSON	In-memory/disk buffer for continuous offline operation when database network access is restricted.

3. Relational Schema Architecture & Data Dictionary

3.1 The analyses Table (Core Forensic Verdict Records)

Column Name	PostgreSQL Type	Nullability	Description & Constraints
id	SERIAL	PRIMARY KEY	Auto-incrementing unique analysis identifier.
content	TEXT	NOT NULL	The raw claim, article text, or URL submitted for verification.
source_type	TEXT	NOT NULL	Input classification: "text", "url", or "image".
prediction	TEXT	NOT NULL	Tri-state forensic determination: "Real", "Fake", or "Misleading".
confidence	INTEGER	NOT NULL	Algorithmic confidence percentage (bounded 0 to 100).
explanation	TEXT	NOT NULL	Executive 1-2 sentence core verdict justification.
keywords	TEXT[]	NULLABLE	PostgreSQL native string array of extracted high-salience topic tags.
detected_source	JSONB	NULLABLE	Extracted publication metadata: { name: string, url: string }.
additional_sources	JSONB	NULLABLE	Corroborating citations array: [{ name, url }].
media_type	TEXT	DEFAULT 'text'	Input modality (text, image base64, external URL).
gemini_verification	JSONB	DEFAULT NULL	Secondary verification dossier and manipulation metrics.
fact_breakdown	JSONB	DEFAULT NULL	Detailed claims list: [{ category, detail, status }].
created_at	TIMESTAMP	DEFAULT NOW()	Audit timestamp of verification creation.

3.2 Conversational Tables (conversations & messages)

Table	Column	Type	Key / References	Description
conversations	id	SERIAL	PRIMARY KEY	Unique conversation session ID.
conversations	title	TEXT	NOT NULL	Auto-generated discussion title or initial topic.
conversations	created_at	TIMESTAMPTZ	DEFAULT NOW()	Timestamp with timezone for global user accuracy.
messages	id	SERIAL	PRIMARY KEY	Unique message identifier.
messages	conversation_id	INTEGER	FOREIGN KEY	References conversations.id with ON DELETE CASCADE.
messages	role	TEXT	NOT NULL	Message actor: "user", "assistant", or "system".
messages	content	TEXT	NOT NULL	Raw message body or streaming markdown tokens.
messages	created_at	TIMESTAMPTZ	DEFAULT NOW()	Chronological ordering index for dialogue reconstruction.

4. Core Code Implementations

4.1 Database Connection Pool & Client Export (lib/db/src/index.js)

```
import { drizzle } from "drizzle-orm/node-postgres";
import pg from "pg";
import * as schema from "../schema/index.js";

const { Pool } = pg;

// Initialize connection pool only when DATABASE_URL environment variable is provided
export const pool = process.env.DATABASE_URL
  ? new Pool({ connectionString: process.env.DATABASE_URL })
  : null;

// Export initialized Drizzle instance or null for graceful offline fallback
export const db = pool ? drizzle(pool, { schema }) : null;

export * from "../schema/index.js";
```

4.2 Relational Drizzle Schema Definition (lib/db/src/schema/analyses.js)

```
import { pgTable, text, serial, integer, timestamp, jsonb } from "drizzle-orm/pg-core";
import { createInsertSchema } from "drizzle-zod";

export const analyses = pgTable("analyses", {
  id: serial("id").primaryKey(),
  content: text("content").notNull(),
  sourceType: text("source_type").notNull(),
  prediction: text("prediction").notNull(),
  confidence: integer("confidence").notNull(),
  explanation: text("explanation").notNull(),
  keywords: text("keywords").array(),
  detectedSource: jsonb("detected_source"),
  additionalSources: jsonb("additional_sources"),
  mediaType: text("media_type").default("text"),
  geminiVerification: jsonb("gemini_verification").default(null),
  factBreakdown: jsonb("fact_breakdown").default(null),
  createdAt: timestamp("created_at").defaultNow(),
});

export const analysisResults = pgTable("analysis_results", {
  id: serial("id").primaryKey(),
  analysisId: integer("analysis_id").notNull(),
  verifiedArticles: jsonb("verified_articles").default([]),
});

export const insertAnalysisSchema = createInsertSchema(analyses).omit({
  id: true,
  createdAt: true,
});
```

4.3 High-Performance SQL Aggregations for Analytics (GET /api/stats)

```
// PostgreSQL Native Query using unnest() for tag frequency calculation
const trending = await db.execute(sql`
  SELECT unnest(keywords) as topic, count(*) as count
  FROM analyses
  GROUP BY topic
  ORDER BY count DESC
  LIMIT 5;
`);

// Parallelized metric counters
const [total, real, fake, misleading] = await Promise.all([
  db.select({ count: sql`count(*)` }).from(analyses),
  db.select({ count: sql`count(*)` }).from(analyses).where(eq(analyses.prediction, "Real")),
  db.select({ count: sql`count(*)` }).from(analyses).where(eq(analyses.prediction, "Fake")),
  db.select({ count: sql`count(*)` }).from(analyses).where(eq(analyses.prediction, "Misleading")),
]);
```

5. Dual-Persistence Resiliency Strategy

A core strength of the SatyaCheck architecture is that database network dropouts, credential rotation delays, or local offline evaluation never prevent the user from completing fact-checks. The system implements a robust **Failover Storage Handler**:

```
const STORAGE_FILE = path.resolve(process.cwd(), "analyses-storage.json");

function loadLocalAnalyses() {
  try {
    if (fs.existsSync(STORAGE_FILE)) {
      return JSON.parse(fs.readFileSync(STORAGE_FILE, "utf-8"));
    }
  } catch {}
  return [];
}

function saveLocalAnalysis(entry) {
  try {
    const list = loadLocalAnalyses();
    list.unshift(entry);
    // Persist latest 100 entries atomically
    fs.writeFileSync(STORAGE_FILE, JSON.stringify(list.slice(0, 100), null, 2), "utf-8");
  } catch (err) {
    console.error("Local storage write error:", err);
  }
}
```

6. PostgreSQL vs. MongoDB Comparison

Evaluation Criterion	PostgreSQL + Drizzle ORM (Selected)	MongoDB / NoSQL Alternative
Data Consistency	Strict ACID guarantees across relational chat & analysis tables.	BASE / Eventual consistency; lacks native multi-table constraints.
Structured + JSON Hybrid	Native JSONB enables indexed JSON querying alongside relational keys.	All data stored as BSON; lacks native relational foreign keys.
Resource Footprint	Lightweight connection pool; zero memory leaks with pure JS driver.	Heavy mongod daemon, elevated RAM overhead for index caches.
Analytical Queries	Fast native SQL GROUP BY, unnest(), and window functions.	Requires complex aggregation pipelines with higher compute cost.

7. Viva & Technical Defense Q&A

Q1: Why did you choose Drizzle ORM over Prisma?

A: Prisma relies on an external Rust binary query engine (over 40MB) which adds cold-start latency and compilation friction. Drizzle is a lightweight, zero-dependency SQL query builder written in pure JavaScript that generates clean SQL matching native queries, running effortlessly in ES Modules.

Q2: Why did you use PostgreSQL instead of MongoDB?

A: SatyaCheck requires relational integrity between conversations and messages (cascade deletes), as well as complex analytical aggregations on keywords and prediction counts. PostgreSQL's native JSONB type gives us the exact flexibility of MongoDB documents while preserving strict foreign key constraints and ACID safety.

Q3: How does the system behave if the PostgreSQL database goes down?

A: The application uses a defensive dual-mode architecture. When DATABASE_URL is absent or the connection throws an exception, all read and write queries automatically redirect to `analyses-storage.json`. The frontend UI continues functioning without throwing HTTP 500 errors.